BASH SCRIPTING

Practice Questions & Hints

A neatly organized study handout covering conditionals, loops, and command-line arguments.

TOPIC 1	Constructs — if, else, elif 12 questions
TOPIC 2	Loops — for, while, until 11 questions
TOPIC 3	Command-line arguments — $1, $2, $#, $@ 11 questions

Study tip: Try writing each script yourself first, then use the hint to guide your approach.

TOPIC 1 | Constructs (if, else, elif)

Q1. Positive, negative, or zero

Create a script that takes a number from the user and checks whether it is positive, negative, or zero.

> ***Hint:*** *Use* `if [ $num -gt 0 ]` *for checking positive numbers and* `elif` *for the other conditions.*

Q2. Voting eligibility

Write a script that asks for a user's age. If the age is 18 or older, print "Eligible to vote", otherwise print "Not eligible".

> ***Hint:*** *Use the* `-ge` *(greater than or equal to) operator inside your conditional brackets.*

Q3. Check whether a file exists

Develop a script that checks if a specific file named `report.txt` exists in the current directory. If it does, print "File found", otherwise print "File missing".

> ***Hint:*** *Use the* `-f` *flag inside the* `if` *condition to check for regular file existence.*

Q4. Check read permissions

Create a script that accepts a filename from the user and checks if the file has read permissions enabled for the current user.

> ***Hint:*** *Use the* `-r` *flag inside your* `if` *statement to test read permissions.*

Q5. Check for an empty string

Write a script that asks the user to enter a string and checks if the entered string is empty or not.

> ***Hint:*** *Use the* `-z` *operator to check if the length of a string is zero.*

Q6. Username authentication

Create a script that asks the user for a username. If the username is "admin", print "Access Granted", otherwise print "Access Denied".

> ***Hint:*** *Use the* `=` *operator to compare strings inside* `[ ]` *or* `[[ ]]`*.*

Q7. Create a backup directory if missing

Write a script that checks if a directory named `backup` exists. If it does not exist, create the directory using `mkdir`.

> ***Hint:*** *Combine the* `!` *(not) operator with the* `-d` *flag to check if a directory is missing.*

Q8. Compare two numbers

Create a script that compares two numbers entered by the user and prints which one is greater, or if they are equal.

> ***Hint:*** *Use* `-gt`*,* `-lt`*, and* `-eq` *for comparing integers.*

Q9. Check for a leap year

Write a script that checks if the current year is a leap year. (Assume the year is hardcoded or read from the user).

> ***Hint:*** *A year is a leap year if it is divisible by 4 but not by 100, or divisible by 400. Use the* `%` *arithmetic operator inside* `(( ... ))`*.*

Q10. Grade from exam score

Create a script that asks for a numerical exam score (0–100) and prints the corresponding grade: A (>=90), B (>=80), C (>=70), or F (<70).

> ***Hint:*** *Use a chain of* `if`*,* `elif`*, and* `else` *statements checking the score from highest to lowest.*

Q11. Password check

Write a script that asks the user for a password. If the password matches "Secret123", print "Welcome", otherwise print "Wrong Password".

> ***Hint:*** *Use string comparison syntax:* `if [ "$password" == "Secret123" ]`*.*

Q12. Delete an empty file

Create a script that checks if a file is empty (has a size of 0 bytes) and deletes it if it is empty.

> ***Hint:*** *Use the `-s` operator, which checks if a file exists and has a size greater than zero; you want the opposite (`! -s`).*

TOPIC 2 | Loops (for, while, until)

Q1. Print numbers 1 to 10

Write a script using a `for` loop to print numbers from 1 to 10 on separate lines.

> ***Hint:*** *Use the sequence expression syntax like* `{1..10}` *inside the* `for` *loop definition.*

Q2. Countdown and blast off

Create a script using a `while` loop that counts down from 5 to 1 and then prints "Blast off!".

> ***Hint:*** *Initialize a variable to 5, and decrement it using* `(( counter-- ))` *inside a loop that runs while* `[ $counter -gt 0 ]`.

Q3. List `.txt` files

Write a script that uses a `for` loop to iterate through all the `.txt` files in the current directory and print their names.

> ***Hint:*** *Use the wildcard pattern* `*.txt` *directly in the* `for file in ...` *loop definition.*

Q4. Repeat until the user types exit

Create a script that uses a `while` loop to continuously ask the user to input a word until they type the word "exit".

> ***Hint:*** *Set the loop condition to check if the input string is not equal to "exit".*

Q5. Use an `until` loop

Write a script using an `until` loop to print numbers from 1 to 5.

> ***Hint:*** *Remember that an* `until` *loop executes until the condition becomes true. Use* `until [ $counter -gt 5 ]`.

Q6. Sum from 1 to 50

Create a script that calculates the sum of numbers from 1 to 50 using a loop.

> ***Hint:*** *Maintain a* `sum` *variable and add the loop index to it during each iteration using* `(( sum += i ))`.

Q7. Multiplication table

Write a script that uses a loop to generate the multiplication table of a number provided by the user (up to 10).

> ***Hint:*** *Multiply the user's input by the loop variable (which runs from 1 to 10) during each iteration.*

Q8. Rename `.jpg` files

Create a script that uses a loop to rename all `.jpg` files in a directory by adding a prefix `old_` to their names.

> ***Hint:*** *Inside a* `for` *loop tracking* `*.jpg`, *use the* `mv` *command to rename* `$file` *to* `old_$file`.

Q9. Read a file with line numbers

Write a script that reads a text file line by line using a `while` loop and prints each line with a line number prefix.

> ***Hint:*** *Combine a* `while read -r line` *loop with file redirection (*`< filename`*) at the very end of the loop block.*

Q10. Calculate factorial

Write a script that uses a loop to find the factorial of a given number.

> ***Hint:*** *Initialize a* `fact` *variable to 1, then use a loop to multiply it by every number down to 1.*

Q11. Print odd numbers

Write a script using a `for` loop to print every odd number between 1 and 20.

> ***Hint:*** *Use the brace expansion step feature like* `{1..20..2}` *to step by 2.*

TOPIC 3 | Command Line Arguments ($1, $2, $#, $@)

Q1. Add two command-line numbers

Write a script that accepts two numbers as command-line arguments and prints their sum.

> ***Hint:*** *Access the first number using `$1` and the second number using `$2`, then use `$(( $1 + $2 ))` to add them.*

Q2. Count arguments

Create a script that prints the total number of arguments passed to it.

> ***Hint:*** *Use the special shell variable `$#` which holds the count of arguments.*

Q3. Print script name and first argument

Write a script that prints the name of the script itself, followed by the first argument passed to it.

> ***Hint:*** *The special variable `$0` stores the name/path of the executing script.*

Q4. Check a file from an argument

Create a script that takes a filename as its first argument and checks if that specific file exists.

> ***Hint:*** *Use `$1` inside your file test condition, like `if [ -f "$1" ]`.*

Q5. Print every argument

Write a script that accepts any number of arguments and prints each argument on a new line using a loop.

> ***Hint:*** *Loop through all arguments by using `for arg in "$@"` or `for arg in "$*"`.*

Q6. Require exactly three arguments

Create a script that expects exactly three arguments. If the user passes more or less than three, print an error message and exit.

> ***Hint:*** *Check `if [ $# -ne 3 ]` at the very beginning of the script.*

Q7. List directories inside a path

Write a script that takes a directory path as the first argument (`$1`) and lists only the directories inside it.

> ***Hint:*** *Loop through `"$1"/*` and check if each item is a directory using the `-d` flag.*

Q8. Require username and password

Create a script that receives a username as `$1` and a password as `$2`. If either argument is missing, print `Usage: script.sh <username> <password>`.

> ***Hint:*** *Check if either variable is empty using `[ -z "$1" ] || [ -z "$2" ]`.*

Q9. Copy a file to a directory

Write a script that accepts two arguments: a source file (`$1`) and a destination directory (`$2`), and copies the file to that directory only if the file exists.

> ***Hint:*** *Verify `[ -f "$1" ]` and `[ -d "$2" ]` before executing the `cp` command.*

Q10. Delete files from arguments

Create a script that takes a list of filenames as arguments and deletes them one by one, printing a confirmation message for each deletion.

> ***Hint:*** *Use a `for file in "$@"` loop to process every filename passed to the command line.*

Q11. Compare two strings

Write a script that takes two string arguments and checks if they are identical.

> ***Hint:*** *No hint was supplied in the original question list.*